TEAM ACTIVITY · FINAL TECHNICAL REPORT

JAMTA

최종 기술 보고서

청소년 전용 학교 커뮤니티 앱의 기술 스택 및 자동 검열 설계

크로스 플랫폼	React Native + TypeScript
서버/API	Python + FastAPI
백엔드 서비스	Supabase(Auth · PostgreSQL · Storage) + Firebase FCM
클라우드 AI	OpenAI Moderation API (텍스트 · 이미지 자동 검열)

보고서 목적

JAMTA 앱의 구현 가능성을 기준으로 크로스 플랫폼, 백엔드 서비스, AI 방식을 비교하고 최종 기술 구성을 제안한다. 오늘의 팀 활동 범위는 데이터베이스 관리, Push 알림, 게시물 · 댓글 · 이미지 자동 검열 설계이다.

제출 정보

구분	기입
팀명 / 팀원	잼타 / 정래준, 우진규, 신하민, 양원준
과목 / 담당교사	고급모바일프로그래밍 / 허준영
제출일	2026년 10월 27일

작성 기준일 2026. 10. 26

01 · PROJECT SCOPE

평가 항목과 최종 선정

과제 요구사항을 빠짐없이 반영한 기술 선택표

평가 영역	검토 내용	최종 선정	선정 이유
크로스 플랫폼	React Native / Flutter	React Native	한 코드베이스로 iOS·Android 개발, TypeScript 활용
서버 기술	Node.js / Python	Python + FastAPI	AI API 연동과 데이터 검증을 서버에서 안전하게 처리
백엔드 서비스	Supabase / Firebase	두 서비스 병행	Supabase는 데이터, Firebase는 Push로 역할 분리
AI 방식	클라우드 / 온디바이스	클라우드 AI	텍스트와 이미지를 하나의 정책으로 검열하고 모델 관리 부담을 줄임

최종 기술 스택

1. React Native 앱이 화면, 입력, 사진 선택, 알림 수신을 담당한다.
2. Python FastAPI가 인증 확인, 입력 검증, 검열 호출, DB 저장 순서를 제어한다.
3. Supabase가 Auth, PostgreSQL, Storage를 제공하며 모든 서비스 데이터의 단일 원본이 된다.
4. Firebase Cloud Messaging(FCM)은 알림 전달에만 사용한다.
5. OpenAI Moderation API가 게시물·댓글 텍스트와 첨부 이미지를 자동 검열한다.

중요한 역할 구분

React Native는 크로스 플랫폼 앱 기술이고 Python은 서버/API 기술이다. 따라서 ‘React Native와 Python을 함께 사용한다’는 말은 두 기술이 같은 화면을 만드는 것이 아니라, 모바일 클라이언트와 서버를 각각 담당한다는 뜻이다.

- 앱에는 Supabase 공개용 anon key만 두고, OpenAI API key와 Firebase 서비스 계정 키는 FastAPI 서버에만 보관한다.
- Firebase Authentication·Firestore·Storage는 이번 설계에서 사용하지 않아 데이터 중복을 피한다.

02 · CROSS PLATFORM

React Native 기반 앱 설계

Flutter와 비교 후 팀 역량 및 구현 범위를 기준으로 선택

항목	React Native	Flutter
언어	TypeScript / JavaScript	Dart
UI 방식	React 컴포넌트와 네이티브 기능 연동	자체 위젯과 렌더링 체계
팀 학습 비용	React 경험을 재사용하기 쉬움	Dart와 위젯 구조 학습 필요
백엔드 연동	Supabase JS SDK, REST 호출	Supabase Flutter SDK, REST 호출
선정 판단	JAMTA 일정과 팀 기술에 적합	구현 가능하지만 이번 과제에서는 미선정

주요 화면과 공통 컴포넌트

화면	핵심 기능	React Native 구현
로그인/회원가입	학교 계정 인증	TextInput, Pressable, Supabase Auth
게시판 선택/피드	학교·지역·학부모 게시판	FlatList, 카드 컴포넌트, 페이지네이션
게시물 상세	좋아요·댓글·신고	Optimistic UI, API 재동기화
게시물 작성	텍스트·사진 최대 5장	이미지 선택, 업로드 진행률, 검열 결과 표시
프로필/활동	내 글·댓글·익명 표시	사용자별 조회, 공개 범위 적용

React Native — 검열 후 게시 요청 예시

```
const submitPost = async () => {
  setLoading(true);
  const body = new FormData();
  body.append('title', title);
  body.append('content', content);
  images.forEach((img) => body.append('images', img));

  const res = await api.post('/posts', body);
  showMessage(res.data.message);   // 허용·차단·재시도 안내
  setLoading(false);
};
```

03 · SERVER API

Python FastAPI의 역할

모바일 앱과 데이터·AI·알림 서비스를 연결하는 보안 경계

책임	처리 내용	서버에서 해야 하는 이유
인증 확인	Supabase JWT 검증, 사용자·학교 확인	위조 요청 및 다른 학교 데이터 접근 차단
입력 검증	제목·본문 길이, 파일 수·형식·크기 검사	앱 검증을 우회한 요청까지 방어
자동 검열	OpenAI Moderation API 호출 및 정책 판정	API key 보호와 정책 일관성 유지
데이터 저장	검열 통과 후 Supabase 트랜잭션 처리	게시물·이미지·로그 불일치 방지
Push 전송	Firebase Admin SDK로 FCM 호출	서비스 계정 키를 앱에 노출하지 않음

FastAPI — 게시물 작성 처리 구조

```
@app.post('/posts')
async def create_post(
    title: str = Form(...), content: str = Form(...),
    images: list[UploadFile] = File(default=[]), user=Depends(current_user)
):
    validate_post(title, content, images)
    decision = await moderate(title, content, images)
    if decision.action != 'ALLOW':
        raise HTTPException(422, detail=decision.user_message)
    post = await save_post_to_supabase(user, title, content, images)
    await write_moderation_log(post.id, decision)
    return {'id': post.id, 'message': '게시물이 등록되었습니다.'}
```

API 경로 제안

Method	경로	용도
POST	/posts	검열 후 게시물 저장
POST	/posts/{id}/comments	검열 후 댓글 저장
POST	/reports	게시물·댓글 신고
POST	/devices/tokens	FCM 토큰 등록·갱신
GET	/notifications	앱 내 알림 목록 조회

04 · BACKEND SERVICES

Supabase + Firebase 역할 분리

두 서비스를 사용하되 같은 데이터를 중복 관리하지 않음

기능	Supabase	Firebase	최종 담당
회원 인증	Auth 지원	Auth 지원	Supabase Auth
관계형 데이터	PostgreSQL·SQL·외래키	Firestore는 문서형	Supabase DB
이미지 저장	Storage + RLS	Cloud Storage 지원	Supabase Storage
실시간 기능	Realtime 지원	Realtime DB/Firestore 지원	필요 기능만 Supabase
Push 알림	직접 전달 기능 아님	FCM 제공	Firebase FCM

데이터 흐름

1. 앱은 Supabase Auth로 로그인하고 Access Token을 발급받는다.
2. 앱이 FastAPI에 Token과 게시물 내용을 전송한다.
3. FastAPI가 사용자·학교 권한과 입력값을 확인한 뒤 OpenAI 검열을 호출한다.
4. 허용된 내용만 Supabase PostgreSQL과 Storage에 저장한다.
5. 댓글·공지처럼 알림 조건이 발생하면 device_tokens를 조회해 FCM을 호출한다.
6. 앱은 Push를 수신하고 notifications 테이블의 상세 데이터를 조회한다.

두 서비스를 함께 사용할 때의 원칙

- 사용자 프로필, 게시물, 메시지, 알림 기록은 Supabase에만 저장한다.
- FCM Token은 Supabase device_tokens에서 관리하고, Firebase에는 메시지 전달만 요청한다.
- 알림 전송 실패는 게시물·댓글 저장 실패로 처리하지 않고 재시도 큐에 남긴다.
- 서비스가 늘어나므로 환경변수, 로그, 장애 지점을 문서화하고 담당자를 정한다.

05 · DATABASE

핵심 데이터 테이블

학교·사용자·게시판·게시물 중심의 핵심 데이터

테이블	주요 열	키 / 설명
schools	id, name, region, domain, created_at	PK id / 학교 기준 정보
profiles	id, school_id, nickname, role, is_anonymous	PK/FK id→auth.users, FK school_id
boards	id, school_id, type, name, is_active	학교·지역·학부모 게시판
posts	id, board_id, author_id, title, content, status, created_at	본문과 게시 상태. author_id는 익명 글도 유지
post_images	id, post_id, storage_path, order_no	Supabase Storage 경로, 글당 최대 5개
post_likes	post_id, user_id, created_at	복합 PK(post_id, user_id)
comments	id, post_id, author_id, parent_id, content, status	parent_id로 답글 표현
comment_likes	comment_id, user_id, created_at	복합 PK(comment_id, user_id)

설계 원칙

- 비밀번호 해시와 로그인 세션은 public 테이블에 저장하지 않고 Supabase Auth에 맡긴다.
- 이미지는 공개 URL보다 storage_path를 저장하고 권한 확인 후 Signed URL을 발급한다.
- 삭제 즉시 행을 없애기보다 status=DELETED 또는 deleted_at을 사용해 신고 조사 기록을 유지한다.
- created_at과 updated_at은 UTC timestamptz로 저장하고 앱에서 현지 시간으로 표시한다.

필수 인덱스

대상	인덱스
피드	posts(board_id, created_at DESC) WHERE status='PUBLISHED'
댓글	comments(post_id, created_at)
사용자 활동	posts(author_id, created_at DESC), comments(author_id, created_at DESC)
미확인 알림	notifications(user_id, is_read, created_at DESC)

05 · DATABASE

운영 데이터 테이블

신고·검열·알림·메시지 운영 데이터

테이블	주요 열	용도
reports	id, reporter_id, target_type, target_id, reason, status	게시물·댓글 신고 및 처리 상태
moderation_logs	id, target_type, target_id, model, flagged, categories, action	AI 결과와 정책 결정 감사 기록
moderation_reviews	id, log_id, reviewer_id, decision, note	애매한 사례의 사람 검토
device_tokens	id, user_id, token, platform, is_active, updated_at	FCM Token의 사용자·기기 연결
notifications	id, user_id, type, title, body, data, is_read	앱 내 알림과 이동 정보
conversations	id, board_id, created_at	대화방 정보
messages	id, conversation_id, sender_id, content, status	메시지 및 검열 상태
notices	id, school_id, title, content, published_at	학교 공지와 알림 기준
banned_words	id, pattern, severity, is_active	빠른 로컬 규칙 검사

RLS(Row Level Security) 정책 예시

- 학생은 자신의 school_id와 연결된 학교 게시판의 공개 글만 조회한다.
- 프로필 수정, 게시물 수정·삭제, device token 변경은 본인 행만 허용한다.
- reports와 moderation_logs는 작성자 본인 또는 관리자만 조회하도록 제한한다.
- Storage 업로드 경로는 user_id/post_id 구조로 만들고 소유자만 쓰기 권한을 가진다.

관계 요약

schools → profiles·boards·notices / boards → posts / posts → post_images·comments·reports / profiles → device_tokens·notifications / posts·comments·messages → moderation_logs → moderation_reviews

06 · AI STRATEGY

클라우드 AI와 온디바이스 AI 비교

평가 항목에는 두 방식을 모두 포함하고, JAMTA에는 클라우드 AI를 선택

항목	클라우드 AI	온디바이스 AI
실행 위치	서버가 외부 AI API 호출	사용자 기기에서 모델 실행
텍스트·이미지	하나의 최신 모델로 함께 분석 가능	각 기능에 맞는 경량 모델 준비 필요
업데이트	제공 모델 업데이트를 즉시 사용	앱 배포 또는 모델 다운로드 필요
성능/기기 차이	서버 환경에서 비교적 일관	기기 성능과 OS에 따라 차이
네트워크	필수	오프라인 가능
개인정보	필요 데이터가 외부 API로 전송됨	기기 밖으로 보내지 않을 수 있음
비용	호출량에 따른 운영 비용	서버 호출 비용은 낮지만 개발·최적화 비용 발생
JAMTA 판단	텍스트·이미지 통합 검열에 적합	오프라인 보조 검사에 향후 검토

OpenAI Moderation API 선정 이유

- omni-moderation-latest가 텍스트와 이미지를 함께 입력받아 하나의 검열 흐름을 만들 수 있다.
- 별도의 분류 모델을 학습·배포하지 않아도 되어 팀 프로젝트의 개발 기간에 적합하다.
- 위험 범주별 결과와 flagged 값을 바탕으로 허용·차단·사람 검토 정책을 구현할 수 있다.
- OpenAI 문서상 Moderation endpoint는 무료이며, API 데이터 정책도 별도 확인할 수 있다.

선정 시 주의점

클라우드 검열은 네트워크 장애와 오판 가능성이 있으므로 AI 결과를 절대적인 판정으로 사용하지 않는다. 고위험은 차단하고, 애매한 사례는 관리자 검토로 보내며, 서비스 오류 시에는 자동 허용하지 않고 임시 저장 또는 재시도를 안내한다.

07 · AUTOMATIC MODERATION

OpenAI Moderation API 적용 설계

게시 전에 텍스트와 이미지를 검사하고 결과를 기록

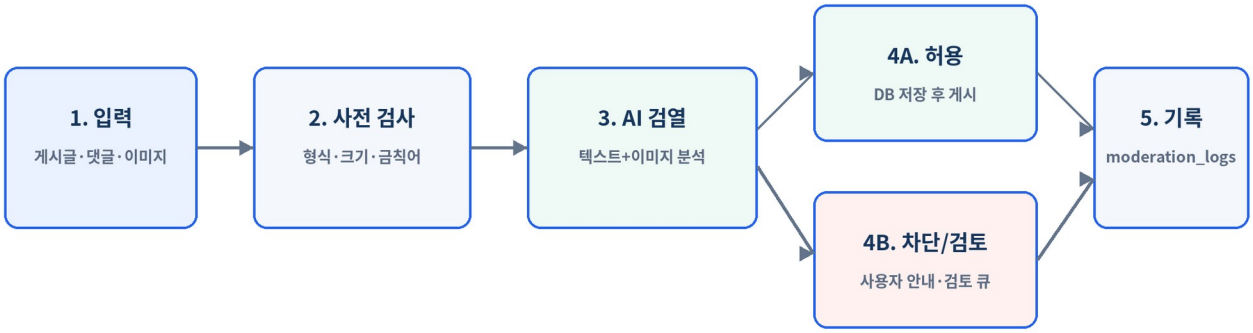


그림 1. 게시물·댓글 자동 검열 처리 흐름

판정	예시 조건	시스템 처리
ALLOW	위험 범주가 정책 기준 미만	게시물 저장 및 정상 등록
BLOCK	성적 미성년자, 자해 방법, 위협적 혐오, 심각한 폭력 등	저장 중단, 안전한 오류 메시지, 로그 기록
REVIEW	문맥이 애매한 괴롭힘·위협·이미지	비공개 임시 저장 후 관리자 검토
ERROR	Timeout, 429, 5xx, 네트워크 오류	자동 허용 금지, 재시도 또는 임시 저장

Python — OpenAI 검열 요청 핵심 예시

```
from openai import OpenAI
client = OpenAI()  # OPENAI_API_KEY는 서버 환경변수

result = client.moderations.create(
    model='omni-moderation-latest',
    input=[
        {'type': 'text', 'text': f'{title} {content}'},
        {'type': 'image_url', 'image_url': {'url': signed_image_url}},
    ],
)
moderation = result.results[0]
action = policy_engine(moderation)  # ALLOW / BLOCK / REVIEW
```

이미지는 검열 전 공개하지 않는다. private 임시 버킷의 짧은 Signed URL 또는 지원되는 입력 형식을 사용하고, 차단된 파일은 즉시 삭제한다. 또한 일반 Moderation API를 불법 아동 성착취물 전용 탐지 시스템으로 간주하지 않는다.

08 · QUALITY & SAFETY

오류 메시지·보안·테스트

사용자는 다음 행동을 알 수 있고, 운영자는 원인을 추적할 수 있어야 함

상황	HTTP/상태	사용자 메시지	시스템 처리
부적절한 내용	422	안전 기준에 맞지 않는 내용이 포함되어 게시할 수 없습니다.	정책 사유 코드 기록, 원문 최소 보관
검토 필요	202/REVIEW	내용을 확인 중입니다. 검토 후 결과를 알려드릴게요.	비공개 저장, 검토 큐 등록
이미지 형식/크기	400	지원하지 않는 파일이거나 용량이 너무 큽니다.	업로드 중단, 제한값 표시
로그인 만료	401	로그인이 만료되었습니다. 다시 로그인해주세요.	Token 갱신 또는 로그인 이동
요청 과다	429	요청이 많습니다. 잠시 후 다시 시도해주세요.	지수 백오프, 중복 제출 방지
검열 서비스 장애	503	안전 확인을 완료하지 못했습니다. 잠시 후 다시 시도해주세요.	자동 허용 금지, 임시 저장 선택
Push 실패	DELIVERY_FAIL	게시물은 저장되었으나 일부 알림 전송이 지연될 수 있습니다.	Token 비활성화·재시도 큐

보안 및 개인정보

- OpenAI와 Firebase 비밀키는 FastAPI 서버의 Secret Manager 또는 환경변수에만 저장한다.
- AI 전송 데이터는 검열에 필요한 제목·본문·이미지로 최소화하고, 이메일·전화번호 등 개인정보는 먼저 제거한다.
- Supabase RLS를 항상 활성화하고 서비스 역할 키는 서버 외부에 노출하지 않는다.
- moderation_logs에는 원문 대신 대상 ID, 모델, 범주, 점수, 결정, 요청 추적 ID를 우선 저장한다.

테스트 계획

구분	확인 항목
정상	한글 게시글·댓글·이미지가 허용되고 DB와 화면에 정상 표시
정책	차단·검토·허용 경계 사례와 오탐/미탐 기록
장애	OpenAI Timeout·429·5xx, Supabase 실패, FCM 실패의 독립 처리
권한	다른 학교·다른 사용자 데이터 접근 및 Storage URL 우회 차단
기기	Android·iOS에서 Push 수신, Token 갱신, 딥링크 이동 확인

09 · CONCLUSION

구현 순서와 결론

기능을 작게 연결한 뒤 검열·알림·권한을 단계적으로 강화

단계	구현 내용	완료 기준
1	Supabase 프로젝트, Auth, 핵심 테이블, RLS	학교별 피드 CRUD와 권한 테스트 통과
2	React Native 로그인·피드·작성 화면	Android/iOS 공통 흐름 동작
3	FastAPI 인증·입력 검증·게시 API	비밀키가 앱에 없고 서버 경유 확인
4	OpenAI Moderation 텍스트·이미지 연동	ALLOW/BLOCK/REVIEW 및 로그 저장
5	Firebase FCM과 device_tokens 연동	댓글·공지 Push 및 실패 재시도
6	오류·보안·성능·정책 테스트	테스트 결과와 한계 문서화

최종 결론

JAMTA는 React Native로 iOS·Android 앱을 함께 개발하고, Python FastAPI를 보안 경계로 둔다. 데이터와 인증·파일은 Supabase에 일원화하며 Firebase는 FCM 알림 전달에만 사용한다. 자동 검열은 OpenAI Moderation API 기반 클라우드 AI를 적용하되, AI 오판과 장애를 고려해 REVIEW 상태와 관리자 검토를 함께 운영한다. 이 구성은 팀의 개발 범위를 줄이면서도 데이터 정합성, 사용자 안전, 확장 가능성을 확보한다.

공식 참고 문서

- OpenAI Moderation guide — https://developers.openai.com/api/docs/guides/moderation
- OpenAI omni-moderation-latest — https://developers.openai.com/api/docs/models/omni-moderation-latest
- OpenAI API data controls — https://developers.openai.com/api/docs/guides/your-data
- React Native FlatList — https://reactnative.dev/docs/flatlist
- FastAPI request files/forms — https://fastapi.tiangolo.com/tutorial/request-forms-and-files/
- Supabase RLS — https://supabase.com/docs/guides/database/postgres/row-level-security
- Supabase Storage access control — https://supabase.com/docs/guides/storage/security/access-control
- Firebase Cloud Messaging — https://firebase.google.com/docs/cloud-messaging
- Firebase server environment — https://firebase.google.com/docs/cloud-messaging/server-environment

※ 가격·API 제한·지원 형식은 변경될 수 있으므로 실제 개발 시작 시 공식 문서를 다시 확인한다.

10 · WHOLE SYSTEM

전체 기술 구조와 연결 관계

마지막 구조도에서 각 기술의 위치와 데이터 이동을 한눈에 확인

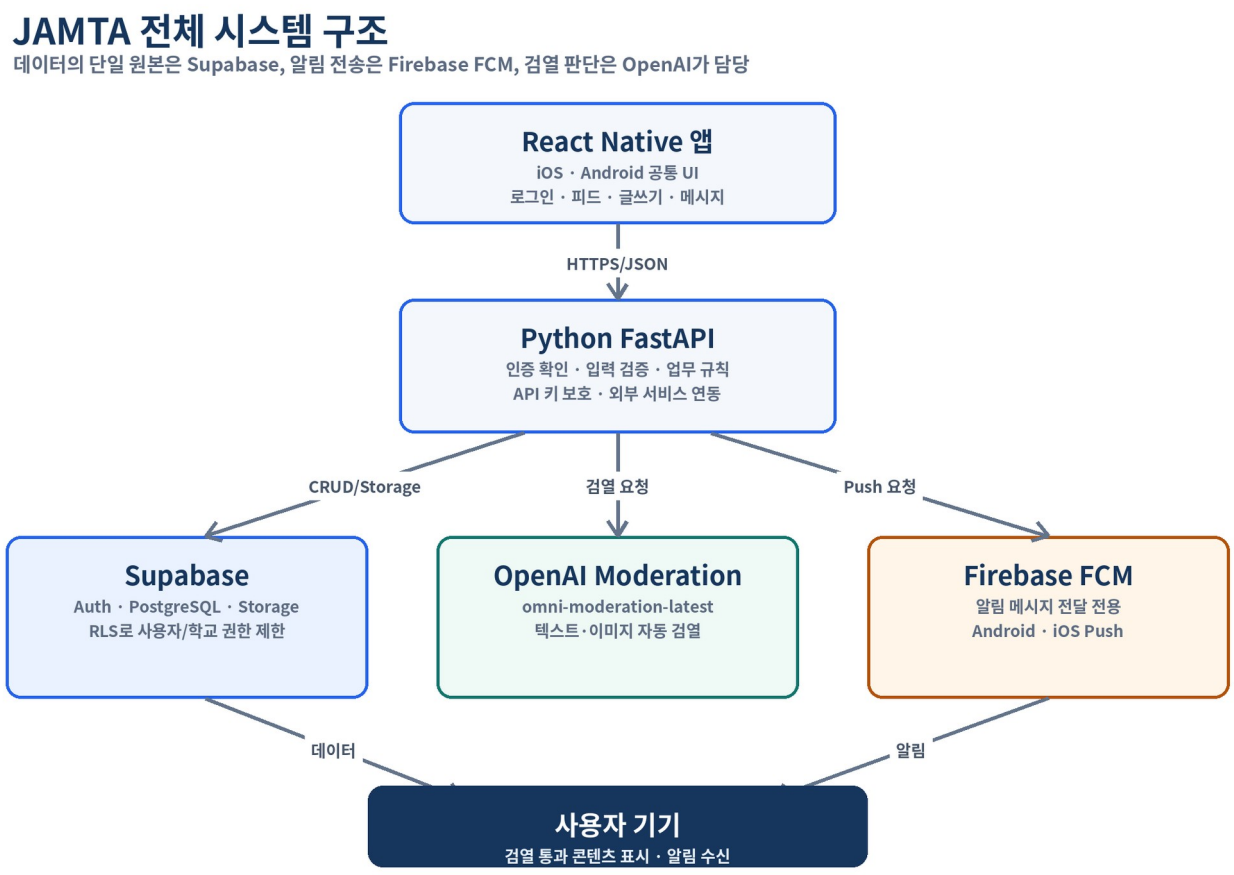


그림 2. JAMTA 전체 기술 구조도

구조도 설명

트리의 시작점은 React Native 앱이다. 모든 중요한 요청은 Python FastAPI를 거쳐 세 갈래로 분기된다. Supabase는 인증·관계형 데이터·이미지의 단일 원본, OpenAI Moderation API는 게시 전 안전 판정, Firebase FCM은 저장 이후 알림 전달을 맡는다. 따라서 데이터가 여러 백엔드에 중복 저장되지 않고, 비밀키도 모바일 앱에 노출되지 않는다.

- 게시 흐름: 앱 → FastAPI → OpenAI 검열 → Supabase 저장 → 필요 시 FCM 알림
- 조회 흐름: 앱 → FastAPI/Supabase → RLS 권한 확인 → 학교별 데이터 표시
- 장애 흐름: 검열 실패 시 게시 보류, FCM 실패 시 데이터는 유지하고 알림만 재시도